

# Why the EPSS-TPG security frontend looks the way it does

A walk-through of the rationale, an inventory of what the security frontend adds, and the questions we are answering or might face

## What this document is, and why it exists

The EPSS-TPG pipeline turns the CVE input text (the NVD description, an LLM-generated summary, or both joined together) into a Text Property Graph and feeds that graph to a multi-view GNN. Most of the graph comes straight from spaCy: tokens, sentences, noun phrases, named entities, and the syntactic edges between them. Once the spaCy layer is in place, a second stage scans the same text with a small library of regular expressions and curated dictionaries and lays a layer of security nodes and edges on top of the linguistic backbone. This second stage is the *security frontend*. Three reasonable questions tend to come up the first time someone reads about it:

1. Why bother with rule-based extraction at all? A neural Named Entity Recognition (NER) model would surely be smarter.
2. What exactly is added on top of the spaCy graph, and what is the basis for picking those particular entities and edges?
3. Which keywords are hard-coded, and do they actually matter for the downstream model?

This note answers all three, and then collects the follow-up questions we tend to come back to whenever the design is challenged or revisited. Every count and every inventory in this document was re-checked against the implementation on the day of writing.

## 1 Why a rule-based security layer is worth the trouble

There are three answers, and they reinforce one another.

**1. The most useful labels are easy to write as rules.** The single most useful thing the security frontend extracts is the CVE identifier itself. A CVE-ID has exactly one shape: the literal letters CVE followed by a year and a sequence number. A regex matches it perfectly on every NVD advisory ever written. The same is true of CWE identifiers, CVSS scores, and semantic version strings. These are syntactically rigid. Training a model to learn them would only add noise: a neural NER might miss one, mislabel one, or hallucinate one that is not there. A regex never does. Spending a few milliseconds per CVE on a regex match is a much better deal than spending a tenth of a parameter budget on relearning what the specification already pins down.

The same logic extends, with somewhat softer precision, to the curated dictionaries. Software product names (OpenSSL, nginx, Log4j) are a closed and slow-moving vocabulary. Common attack-vector phrases (*remote*, *unauthenticated*, *user-supplied input*) and impact phrases (*remote code execution*, *denial of service*, *privilege escalation*) recur across thousands of advisories with nearly identical surface forms. A hand-maintained dictionary captures them with high precision and is small enough that an analyst can audit it in an afternoon.

**2. A rule-produced graph can be audited end-to-end.** Every entity created by the security frontend carries a tag identifying which dictionary or regex produced it, together with the character offsets in the original text. When the GNN flags a CVE as high-risk and a human asks “which evidence in the text drove that?”, the answer is reachable: it was these specific entity nodes, each created by this specific match. A learned extractor would make the same decisions, but the audit trail would disappear. That trade-off only becomes attractive once recall becomes a real bottleneck, and a separate hybrid layer based on contextual-embedding similarity addresses recall without giving up the audit trail.

**3. Typed security edges are what give the encoder a fifth view.** The multi-view GGNN encodes the graph one edge family at a time and then fuses the views by attention. Without typed security edges, every security relation would collapse into a generic “entity-to-entity” bucket and the encoder would have no way to isolate a security view at all. The edge typing is not just bookkeeping; it is what makes the fifth view of the encoder possible.

## 2 Why spaCy or SecBERT alone are not enough

This is the question that comes up most often.

spaCy’s English NER produces tags from the eighteen-label OntoNotes 5 scheme: CARDINAL, DATE, EVENT, FAC, GPE, LANGUAGE, LAW, LOC, MONEY, NORP, ORDINAL, ORG, PERCENT, PERSON, PRODUCT, QUANTITY, TIME, WORK\_OF\_ART. None of those mean anything to a security model. Concretely:

- CVE-2024-1234 becomes a plain TOKEN. spaCy has no schema slot for it.
- *OpenSSL*, *nginx* and *Log4j* may or may not be tagged ORG or PRODUCT depending on the surrounding context. Even when they are, there is no SOFTWARE label that the downstream GNN can attend to as its own slot.
- *CVSS 9.8*, *critical severity*, *buffer overflow*, *RCE* carry no special meaning to a generic parser.
- spaCy never emits security *relations* (CVE → CWE, CVE → Software, Attack-Vector → Impact). The whole typed security edge layer would be missing.

SecBERT is closer to the right tool, but it is a contextual embedding model, not a tagger. The pipeline uses it extensively: it embeds every token, sentence, and entity node with a 768-dimensional contextual vector, and that is where most of the model’s semantic capacity comes from. What it does not do on its own is decide that “this span is a CVE-ID, that span is a software product, that other span is an attack vector”. That classification step is still the job of either a tagger (neural or rule-based) or a downstream model trained on labelled spans. Building a labelled-span dataset for CVE descriptions is its own multi-month project. Rules give roughly ninety percent of the signal for almost no annotation cost, and the hybrid fallback adds a contextual-embedding step for the rest.

So the layered design, in plain English, is this: spaCy provides the syntactic backbone, SecBERT provides the contextual semantics on every node, the security frontend provides the security-specific labels and edges, and the hybrid layer recovers the entities the rules missed.

## 3 What the security frontend actually adds to the graph

### 3.1 Twelve security node types in the schema, ten of them populated

The schema declares twelve security node types:

CVE\_ID, CWE\_ID, SOFTWARE, VERSION, COMPONENT, CODE\_ELEMENT, ATTACK\_VECTOR, IMPACT, REMEDIATION, THREAT\_ACTOR, VULN\_TYPE, SEVERITY

Ten of these are populated by the extractors: CVE\_ID, CWE\_ID, SOFTWARE, VERSION, CODE\_ELEMENT, ATTACK\_VECTOR, IMPACT, REMEDIATION, VULN\_TYPE, SEVERITY. The remaining two (COMPONENT and THREAT\_ACTOR) are reserved slots in the schema. They are not yet emitted, because the corresponding vocabularies have not been curated. The honest count is “ten populated security node types out of twelve declared”.

In the graph itself, every security entity is stored as an ordinary entity node. The “security-ness” lives in two properties: a top-level label (CVE\_ID, SOFTWARE, and so on) and a narrower tag (*rce*, *remote*, *buffer\_overflow*, and so on). The unified label space prefixes the security labels with SEC\_\* so that the GNN sees a single integer slot per security type.

### 3.2 Ten security edge types, all of them emitted

The schema also declares ten typed security edges, and the security-relations stage emits each one whenever both endpoint entities exist in the same document (Table 1). The basis column points at the canonical security-data field each edge mirrors.

### 3.3 Eight regex patterns and one regex list

The security frontend defines eight specific regex patterns plus the remediation pattern list (Table 2).

### 3.4 Five keyword dictionaries

Table 3 lists the five keyword dictionaries, their current sizes, and a sample of the narrow tags each one emits. The counts come from a direct tally of the literal entries in the implementation.

## 4 Why these specific entities and edges, and not others

The inventory above is opinionated, and we are aware of that. The following paragraphs explain each major design choice, framed in terms of what the downstream exploitation-prediction task actually cares about.

**Entities are picked for what the model can act on, not for what is easy to extract.** VULN\_TYPE (what the bug is) and IMPACT (what the attacker gets) are central because they predict exploitation directly: memory-corruption bugs and RCE outcomes correlate strongly with high EPSS scores; denial-of-service outcomes correlate much less. ATTACK\_VECTOR captures reachability, where network- and remote-without-authentication wins. SOFTWARE and VERSION let the model attend to product-specific patterns; a Log4j-class bug is not an obscure-library bug. REMEDIATION is a coarse proxy for “is a patch available”, which shifts the exploitation timeline. SEVERITY and CVSS scores are the assessor’s qualitative band, complementing the numerical CVSS features in the tabular branch. CVE\_ID and CWE\_ID anchor the description to the NVD and CWE ontologies; even one CVE identifier in the text tells the model it is reading a vulnerability description.

**Edges connect entities that CVE descriptions actually relate.** The ten security edges are not invented. Each one mirrors a field that already exists in NVD or CVSS. “CWE to Mitigation”, for example, was deliberately not added as a typed edge, because CVE advisories almost never write a mitigation in terms of a CWE category; they write it in terms of the specific CVE. The security

| Edge              | Source        | Target        | Basis (canonical security data)                                                                                           |
|-------------------|---------------|---------------|---------------------------------------------------------------------------------------------------------------------------|
| SEC_AFFECTS       | CVE           | SOFTWARE      | NVD ships an explicit affected-products list per CVE.                                                                     |
| SEC_HAS_VERSION   | SOFTWARE      | VERSION       | NVD ships affected version ranges per product.                                                                            |
| SEC_CLASSIFIED_AS | CVE           | CWE           | NVD assigns one or more CWE weakness classes to every CVE.                                                                |
| SEC_LOCATED_IN    | VULN_TYPE     | CODE_ELEMENT  | Where the weakness sits in code, for example a buffer overflow inside a string-copy primitive.                            |
| SEC_EXPLOITED_BY  | CVE           | ATTACK_VECTOR | The attack-vector axis of CVSS (network, local, physical, adjacent).                                                      |
| SEC_CAUSES        | ATTACK_VECTOR | IMPACT        | The impact axis of CVSS (confidentiality, integrity, availability) mapped to RCE, DoS, information disclosure, and so on. |
| SEC_MITIGATED_BY  | CVE           | REMEDIATION   | Mitigation, upgrade, or patch language at the end of most advisories.                                                     |
| SEC_USES_FUNCTION | CVE           | CODE_ELEMENT  | Direct mention of a vulnerable function or construct in the description.                                                  |
| SEC_THREATENS     | ATTACK_VECTOR | SOFTWARE      | Network- or remote-reachable vectors threaten the product they can reach.                                                 |
| SEC_HAS_SEVERITY  | CVE           | SEVERITY      | CVSS's qualitative bands (critical, high, medium, low, informational).                                                    |

Table 1: The ten typed security edges and the canonical security-data field each one mirrors. The full edge vocabulary the GNN sees is therefore 13 base edge types plus these 10 security edge types, for 23 in total when the security view is active.

| Pattern                 | What it matches                                                                                                                                                                                                                                                                                                                                             |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CVE identifier          | <b>CVE-YYYY-NNNN+</b> , including the six Unicode hyphen variants in the range U+2010 to U+2015 that LLM-generated summaries often emit.                                                                                                                                                                                                                    |
| CWE identifier          | <b>CWE-NNNN</b> .                                                                                                                                                                                                                                                                                                                                           |
| Semantic version        | Versions of the form <b>1.2.3</b> , <b>1.0.0-rc1</b> , or <b>4.18.0-372</b> . A context filter rejects matches that look like CVSS scores.                                                                                                                                                                                                                  |
| CVSS score              | <i>CVSS 9.8</i> , <i>CVSS:9.8</i> , <i>CVSS v3 9.8</i> , <i>9.8/10</i> .                                                                                                                                                                                                                                                                                    |
| Severity with context   | Adjective plus security-context noun ( <i>critical severity</i> , <i>high CVSS score</i> ). The mandatory second token blocks false positives like “high availability”.                                                                                                                                                                                     |
| Bare-severity fallback  | A bare severity word followed by punctuation, fired only when a security keyword is within sixty characters.                                                                                                                                                                                                                                                |
| Code element            | Sixteen dangerous C-standard-library and scripting function names ( <i>strcpy</i> , <i>strcat</i> , <i>sprintf</i> , <i>gets</i> , <i>scanf</i> , <i>memcpy</i> , <i>memmove</i> , <i>malloc</i> , <i>free</i> , <i>printf</i> , <i>fprintf</i> , <i>eval</i> , <i>exec</i> , <i>system</i> , <i>popen</i> , <i>fork</i> ) plus the generic “name()” shape. |
| Vendor-product fallback | Capitalised <i>Vendor Product</i> fragments immediately before a version number. Catches niche products absent from the curated software dictionary.                                                                                                                                                                                                        |
| Remediation actions     | A list of sixteen small patterns mapped to the tags <i>upgrade</i> , <i>patch</i> , <i>update</i> , <i>workaround</i> , <i>mitigation</i> , <i>disable_feature</i> , <i>restrict_access</i> , <i>network_block</i> .                                                                                                                                        |

Table 2: Regex patterns in the security frontend. Each pattern is paired, where appropriate, with a context filter that rejects false positives in surrounding sentences.

| Dictionary             | Phrases | Examples and narrow tags                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Software product names | 127     | apache, nginx, openssl, openssh, linux kernel, mysql, postgresql, log4j, struts, jackson, fastjson, vmware, vcenter, esxi, fortios, pan-os, asa, big-ip, moveit, confluence, jira, exchange server, ...                                                                                                                                                                                                                                |
| Attack vectors         | 47      | <i>remote</i> , <i>network</i> , <i>local</i> , <i>physical</i> , <i>adjacent</i> , <i>user input</i> , <i>crafted input</i> , <i>malicious input</i> , <i>attacker</i>                                                                                                                                                                                                                                                                |
| Impact types           | 51      | <i>rce</i> , <i>dos</i> , <i>info disclosure</i> , <i>privesc</i> , <i>auth bypass</i> , <i>sqli</i> , <i>xss</i> , <i>csrf</i> , <i>ssrf</i> , <i>uaf</i> , <i>buffer overflow</i> , <i>integer overflow</i> , <i>heap overflow</i> , <i>stack overflow</i> , <i>path traversal</i> , <i>data breach</i> , <i>data exposure</i>                                                                                                       |
| Vulnerability types    | 47      | <i>buffer overflow</i> , <i>buffer over-read</i> , <i>use after free</i> , <i>double free</i> , <i>race condition</i> , <i>command injection</i> , <i>sqli</i> , <i>xss</i> , <i>csrf</i> , <i>ssrf</i> , <i>xxe</i> , <i>deserialization</i> , <i>unsafe reflection</i> , <i>path traversal</i> , <i>missing authentication</i> , <i>weak cryptography</i> , <i>hardcoded credentials</i> , <i>prototype pollution</i> , <i>redos</i> |
| Code constructs        | 44      | serialization, deserialization, reflection api, object graph, template engine, yaml/xml/json/url parser, request handler, session manager, session token, prepared statement, csrf token, authorisation check, ...                                                                                                                                                                                                                     |

Table 3: Sizes of the five keyword dictionaries used by the security frontend.

frontend is therefore deliberately conservative: ten edge types, all with a clear data-model origin, all checkable against an NVD record.

## 5 Questions we are answering, and ones we expect to face

### Q. How accurate is this? Has anyone actually measured the precision and recall?

The high-precision regex extractors (CVE-ID, CWE-ID, semantic version, CVSS) are designed to achieve effectively one-hundred-percent precision on NVD-formatted text by construction; the shapes they match are syntactic. The dictionary extractors trade some precision for coverage, but each one is anchored on word boundaries and uses longest-first matching to avoid the classic “git inside github” failure mode. A labelled benchmark of human-annotated CVE spans is not yet available, so a precise P/R number against ground truth cannot be quoted today. What can be said is that the extractor is deterministic and auditable, so any failure can be traced to a specific rule and fixed in one place. Building a labelled test set is the sensible next step the day such a benchmark is needed.

### Q. Doesn’t the version regex confuse CVSS scores like 9.8 with software versions?

That is the failure mode the version extractor is specifically guarded against. The version pattern is paired with a context filter that rejects any candidate whose surface shape looks like a CVSS score *and* whose surrounding sentence (within plus or minus eighty characters) mentions *CVSS*, *base score* or *severity score*. A second short-window guard rejects the “9.8/10” idiom. Severity uses a similar two-stage design: a precise pattern that requires a security-context noun runs first, and the bare-severity fallback only fires when a security keyword sits within sixty characters. The general principle is that every loose pattern is paired with a context filter.

### Q. Why not just train a neural NER on CVE descriptions and skip the rules entirely?

Three reasons. First, there is no widely-used labelled corpus of CVE spans, so the team would have to annotate one, which is months of work. Second, the dominant entities (CVE-IDs, CWE-IDs, versions, CVSS scores) are exactly the cases where rules win by construction; a neural NER would have to learn them and could hallucinate. Third, the audit-trail story matters. Anyone asking “why did this CVE receive a high predicted score?” can follow the chain back to specific entity nodes and the specific regex match that produced each one. With a neural NER, the chain breaks at the model boundary. The combination of high-precision rules plus a contextual-embedding fallback gives most of the recall benefit without giving up the audit trail.

### Q. Where did the dictionary entries come from in the first place?

The vocabulary grew empirically while running the pipeline over the social-media dataset, the Megavul dataset, and the NVD corpus, and inspecting which entity types under-fired. The software list started from the products that recur most often in NVD descriptions and was extended whenever a recurring CVE family (Telerik UI, FortiOS, vCenter, Confluence, MoveIt, Log4j, and the like) was missed. The impact, vulnerability-type and attack-vector dictionaries came from standard CWE and OWASP terminology with duplicates merged. The remediation patterns came from inspecting the closing sentences of advisories. The vendor-product fallback regex catches niche proprietary products that never reach the curated dictionary.

### Q. What happens when a phrase like “buffer overflow” fits two dictionaries at once?

The three keyword extractors that compete for shared phrases (VULN\_TYPE, IMPACT, ATTACK\_VECTOR) share a single list of already-claimed character ranges. VULN\_TYPE runs first, because it is the most specific tag. IMPACT runs next, and ATTACK\_VECTOR last. The first extractor to claim a character range wins. About fourteen phrases overlap between IMPACT and VULN\_TYPE; they all end up tagged as VULN\_TYPE.

**Q. Is the security view actually used in the main training runs, or is this just on paper?**

The nineteen-run social-media ablation and the fifteen-run Megavul ablation reported elsewhere in this work were trained without the security-edge view, on the base thirteen-edge graph. The security-edge view was added to the pipeline later. The NVD/KEV temporal runs and the most recent Megavul runs do use it. The honest framing for a paper is therefore: the security view is an opt-in fifth view; ablations are reported both ways. The figures that say “four views (or five with security)” reflect this.

**Q. How much upkeep does this need? When NVD adds a new CWE class, does anything have to change?**

Almost nothing. The CWE regex matches any *CWE-NNNN*-shaped string regardless of which CWE number it carries, so new CWE classes appear in the graph automatically. The software dictionary is the one piece that ages: when a new vendor product starts showing up in advisories and the vendor-product fallback misses it, an analyst adds the name to the dictionary. This amounts to a few minutes per quarter in practice.

**Q. What about CVE descriptions written in languages other than English?**

The current extractors assume English text. The dictionaries are in English and the spaCy model is the English small-web model. CVE descriptions in NVD are English by policy. Translated text from the LLM-summary pipeline is also English-only. If a non-English corpus is ever ingested, a language-detection step plus a per-language frontend would be the right pattern, and the schema already supports stacking frontends.

**Q. If the rules already do most of the work, what is the hybrid layer for?**

The hybrid layer runs the rule-based extractor first, then uses contextual-embedding similarity to find spans that are close in embedding space to existing security entities but that the rules missed. Rule entities win on overlap, so the audit trail stays intact for the rule-detected entities. This is the recall safety net: it picks up novel software names and re-phrasings the dictionaries do not yet know about.

**Q. Does upper-case versus lower-case matter? Will the extractor miss “OPENSSL” written in all capitals?**

No. Every regex is case-insensitive and every dictionary lookup is performed against the lowercased version of the text. The original casing is preserved on the resulting entity node so the display still reads naturally.

**Q. Why does any of this belong in a paper? Isn’t it just pre-processing?**

Two reasons. First, the multi-view encoder’s security view only exists because of these typed edges; turning it off collapses the architecture from five views to four. The extractor is therefore part of the model description, not just a data loader. Second, the model is interpretable in part because this pre-processing is auditable. A paper that simply “fed text to a big GNN and reported numbers” would not need this section; this one is not that paper.

## 6 Empirical evidence: the GPT ablation (five variants)

To check whether the security frontend actually earns its keep against the new 15-run social-media baseline (Section 8), the model was re-trained on the same five GPT text-source variants (*D*, *SMP*, *S\_git*, *S\_cvss*, *ALL*) with the security frontend disabled. In this ablation the graph is built by the plain spaCy frontend only, so every security entity node and every typed security edge is removed; the

NOSEC runs are kept in a separate cache so they do not collide with the WITH-sec runs. Everything else (architecture, hyperparameters, data splits, soft EPSS regression target, EPSS feature removed from the tabular branch to prevent leakage) is identical. The test sets are identical between the two halves of each comparison (Table 4), so the numbers are directly comparable.

| Variant       | Test CVEs | Positives | Positive % |
|---------------|-----------|-----------|------------|
| <i>D</i>      | 1 385     | 215       | 15.5 %     |
| <i>SMP</i>    | 1 383     | 214       | 15.5 %     |
| <i>S_git</i>  | 339       | 58        | 17.1 %     |
| <i>S_cvss</i> | 1 385     | 215       | 15.5 %     |
| <i>ALL</i>    | 1 385     | 215       | 15.5 %     |

Table 4: Test-set composition for the five GPT variants (soft EPSS target). The same test split is used for both the WITH-security and NOSEC runs of each variant. *SMP* loses two CVEs to an empty social-media post field; *S\_git* evaluates on the subset of CVEs with a non-empty GitHub-references summary.

Table 5 shows the headline ranking metric (test PR-AUC) for each variant, side by side, and Table 6 expands the comparison to the full six-metric table.

| Variant          | WITH security | NOSEC                                   | $\Delta$ PR-AUC   | Winner |
|------------------|---------------|-----------------------------------------|-------------------|--------|
| <i>D</i>         | <b>0.8394</b> | 0.5416                                  | −0.2978 (−35.5 %) | WITH   |
| <i>SMP</i>       | <b>0.8321</b> | 0.5738                                  | −0.2583 (−31.0 %) | WITH   |
| <i>S_git</i>     | 0.7553        | <b>0.7846</b>                           | +0.0293 (+3.9 %)  | NOSEC  |
| <i>S_cvss</i>    | 0.4916        | <b>0.5819</b>                           | +0.0903 (+18.4 %) | NOSEC  |
| <i>ALL</i>       | <b>0.7878</b> | 0.5286                                  | −0.2592 (−32.9 %) | WITH   |
| WITH wins        |               | 3 of 5 variants (D, SMP, ALL)           |                   |        |
| NOSEC wins       |               | 2 of 5 variants (S_git, S_cvss)         |                   |        |
| Average $\Delta$ |               | −0.1391 PR-AUC across the five variants |                   |        |

Table 5: GPT ablation, headline ranking metric (test PR-AUC). The security frontend wins three of five variants by large margins (26–30 PR-AUC points) and loses two by smaller margins (3–9 points); the average drop without the security frontend is 13.9 PR-AUC points across the five variants.

**What the numbers say.** The security frontend is doing serious work on the variants that consume canonical CVE-description language or the raw social-media post text. On the description-only variant, removing the security frontend cuts test PR-AUC by 29.8 points ( $0.8394 \rightarrow 0.5416$ ); on the combined-everything variant, the drop is 25.9 points ( $0.7878 \rightarrow 0.5286$ ). The new *SMP* variant shows the most dramatic NOSEC failure mode: PR-AUC drops by 25.8 points and recall collapses from 0.8832 to 0.1355, meaning the model trained on raw social-media text without security signals essentially stops firing positives. These are large effects for a fixed-architecture ablation and are the clearest evidence yet that the security overlay is what lets the GNN turn unstructured text into a reliable exploit-prediction signal.

**Where the security frontend does not help, and why that makes sense.** The two NOSEC wins occur on text variants that already encode the security information in their own way and that have sparse or redundant security overlays:

- *S\_git* is built from a GitHub-URL summary that paraphrases commit messages. Commit text rarely uses the formal CVE phrasing the regex and dictionary layer relies on (“buffer overflow in OpenSSL 1.0.1f”); it talks about code, diffs, and bug fixes. The security frontend contributes a sparse overlay here (only 14.9 mean SEC\_\* edges per graph, against the description’s 4.6 baseline

| Variant       | Metric    | WITH          | NOSEC         | $\Delta$ | Winner |
|---------------|-----------|---------------|---------------|----------|--------|
| <i>D</i>      | PR-AUC    | <b>0.8394</b> | 0.5416        | −0.2978  | WITH   |
| <i>D</i>      | ROC-AUC   | <b>0.9400</b> | 0.8692        | −0.0708  | WITH   |
| <i>D</i>      | F1        | <b>0.8095</b> | 0.5907        | −0.2188  | WITH   |
| <i>D</i>      | Precision | <b>0.8293</b> | 0.5050        | −0.3243  | WITH   |
| <i>D</i>      | Recall    | <b>0.7907</b> | 0.7116        | −0.0791  | WITH   |
| <i>D</i>      | Brier ↓   | <b>0.0493</b> | 0.1211        | +0.0718  | WITH   |
| <i>SMP</i>    | PR-AUC    | <b>0.8321</b> | 0.5738        | −0.2583  | WITH   |
| <i>SMP</i>    | ROC-AUC   | <b>0.9549</b> | 0.9000        | −0.0549  | WITH   |
| <i>SMP</i>    | F1        | <b>0.7590</b> | 0.2257        | −0.5333  | WITH   |
| <i>SMP</i>    | Precision | 0.6655        | <b>0.6744</b> | +0.0089  | NOSEC  |
| <i>SMP</i>    | Recall    | <b>0.8832</b> | 0.1355        | −0.7477  | WITH   |
| <i>SMP</i>    | Brier ↓   | <b>0.0657</b> | 0.1019        | +0.0362  | WITH   |
| <i>S_git</i>  | PR-AUC    | 0.7553        | <b>0.7846</b> | +0.0293  | NOSEC  |
| <i>S_git</i>  | ROC-AUC   | <b>0.9250</b> | 0.9197        | −0.0053  | WITH   |
| <i>S_git</i>  | F1        | <b>0.8065</b> | 0.5579        | −0.2486  | WITH   |
| <i>S_git</i>  | Precision | <b>0.7576</b> | 0.4015        | −0.3561  | WITH   |
| <i>S_git</i>  | Recall    | 0.8621        | <b>0.9138</b> | +0.0517  | NOSEC  |
| <i>S_git</i>  | Brier ↓   | <b>0.0600</b> | 0.1888        | +0.1288  | WITH   |
| <i>S_cvss</i> | PR-AUC    | 0.4916        | <b>0.5819</b> | +0.0903  | NOSEC  |
| <i>S_cvss</i> | ROC-AUC   | 0.8384        | <b>0.8718</b> | +0.0334  | NOSEC  |
| <i>S_cvss</i> | F1        | 0.4541        | <b>0.5701</b> | +0.1160  | NOSEC  |
| <i>S_cvss</i> | Precision | 0.5419        | <b>0.5636</b> | +0.0217  | NOSEC  |
| <i>S_cvss</i> | Recall    | 0.3907        | <b>0.5767</b> | +0.1860  | NOSEC  |
| <i>S_cvss</i> | Brier ↓   | 0.1051        | <b>0.0975</b> | −0.0076  | NOSEC  |
| <i>ALL</i>    | PR-AUC    | <b>0.7878</b> | 0.5286        | −0.2592  | WITH   |
| <i>ALL</i>    | ROC-AUC   | <b>0.9390</b> | 0.8624        | −0.0766  | WITH   |
| <i>ALL</i>    | F1        | <b>0.7621</b> | 0.5568        | −0.2053  | WITH   |
| <i>ALL</i>    | Precision | <b>0.6962</b> | 0.5342        | −0.1620  | WITH   |
| <i>ALL</i>    | Recall    | <b>0.8419</b> | 0.5814        | −0.2605  | WITH   |
| <i>ALL</i>    | Brier ↓   | <b>0.0661</b> | 0.1013        | +0.0352  | WITH   |

Table 6: Full six-metric WITH versus NOSEC comparison for the five GPT variants on the new 15-run social-media baseline. Bold marks the better value. Brier is lower-is-better; all other metrics are higher-is-better. Per-metric WITH-win tallies (out of five variants): PR-AUC 3, ROC-AUC 4, F1 4, Precision 3, Recall 3, Brier 4. The single all-NOSEC sweep is *S\_cvss*; the single all-WITH sweep is *ALL*.

and *ALL*’s 57.7; see Table 11), and the NOSEC model edges past it on PR-AUC and recall while still losing badly on precision (0.40 vs 0.76) and F1.

- *S\_cvss* is built from a CVSS-metrics summary, which already encodes attack vector, attack complexity, privileges, scope, and impacts in a fixed CVSS vector format. The security frontend’s overlapping tags become redundant duplicates of information the contextual embedding already captures: *S\_cvss* produces the lightest security overlay of all five variants (only 0.77 mean SEC\_\* edges per graph; see Table 11). Stripping that thin overlay actually *helps* the model here: NOSEC wins on every one of the six metrics for this variant.

**The calibration story is nearly unanimous.** Looking only at PR-AUC understates the case. The Brier score (lower is better) is worse without the security frontend on four of five variants, and the worst case is the GitHub-URL summary: Brier balloons from 0.0600 with security to 0.1888 without, a threefold degradation in calibration. F1 also drops on four of five variants without security, with the catastrophic *SMP* F1 collapse ( $0.7590 \rightarrow 0.2257$ ) topping the list. Precision falls hard on four of five variants (down 32 points on *D*, down 36 points on *S\_git*, down 16 points on *ALL*) while recall on those same four either falls together with it or stays roughly flat, meaning the NOSEC model is not just wider but truly poorly calibrated. The security frontend’s role is partly to give the model a strong, narrow signal it can rely on; strip that signal and the calibration goes with it.

**Verdict.** On the GPT block under the new 15-run baseline the security frontend wins clearly: the WITH-security model is better on the three most informative variants (*D*, *SMP*, *ALL*) by 26–30 PR-AUC points, and the two losses (*S\_git*, *S\_cvss*) are smaller (3 and 9 PR-AUC points respectively) and happen on text sources where the security frontend structurally has little to contribute (commit-message paraphrase, CVSS-vector restatement). The average PR-AUC across the five variants drops by 13.9 points when the security frontend is removed — a larger effect than the 10.2-point average we reported on the earlier *S\_all*-based baseline, mostly because the new *SMP* variant exposes a 26-point WITH-vs-NOSEC gap that the LLM-curated *S\_all* did not. Precision, F1, recall, and Brier all degrade in three or four of the five variants without the security frontend.

The same ablation should still be repeated on the remaining datasets (the 15 Megavul variants and the remaining two social-media LLMs) before a final conclusion is reported. The GPT block is the first slice on the new baseline and is strongly in favour of keeping the security frontend; the per-LLM trends summarised in Section 7 (Megavul) and Section 8 (social media) indicate that Gemma and Mistral are expected to follow the same pattern.

## 7 Per-LLM characterisation of the Megavul input graphs

This section answers the natural reviewer question for the commit-based Megavul corpus: *the security frontend treats every input the same way, so do the resulting graphs actually look different across LLM summarisers, and does the security overlay fire differently?* The social-media half of this analysis is now consolidated into Section 8, which hosts the 15-run social-media baseline end-to-end (graph dimensions, security overlay, and downstream metrics).

The 15 Megavul (LLM  $\times$  variant) datasets used here were scanned with two complementary passes:

- The processed PyG cache was opened directly and the mean number of nodes and edges per graph were computed from the per-graph slice offsets. This is cheap because no parsing is needed, only tensor reads.
- Each labelled record’s input text was re-run through the rule-only security pipeline, with the security-relations layer enabled, to count how often each of the ten security entity types and the ten typed SEC\_\* edges fires.

**Cross-LLM sanity check.** On the description-only variant (*D*) every graph is built from the same CVE description text, so the per-LLM numbers should agree exactly. They do: every Megavul *D* run reports the same 108.6 nodes / 357.6 edges per graph and the same 75.0 % of CVEs producing at least one SEC\_\* edge. This confirms the extraction is deterministic and that any cross-LLM difference observed below is genuinely the LLM summariser’s output, not pipeline noise.

**Headline numbers per Megavul LLM.** Table 7 shows the three Megavul blocks on the most informative variant (*ALL*, which feeds the model the CVE description plus every available LLM summary). The columns are: mean nodes per graph, mean edges per graph, the share of CVEs producing at least one SEC\_\* edge, and the mean number of SEC\_\* edges per graph.

| LLM     | Mean nodes | Mean edges | %CVEs $\geq 1$ SEC | Mean SEC edges | Mean sec. entities |
|---------|------------|------------|--------------------|----------------|--------------------|
| GPT     | 848        | 2,858      | 99.8 %             | 57.91          | 22.37              |
| Gemma   | 640        | 2,065      | 100.0 %            | 38.90          | 19.40              |
| Mistral | 988        | 3,201      | 99.9 %             | 98.83          | 29.06              |

Table 7: Per-LLM characterisation of the Megavul commit-based graphs on the *ALL* variant. Mean nodes and mean edges per graph come from a direct read of the processed PyG cache; the SEC\_\* firing rate and the mean counts of typed security edges and security entities come from re-running the rule-only security pipeline over each dataset’s labelled records. “%CVEs  $\geq 1$  SEC” is the *share* of test CVEs whose graph contains at least one SEC\_\* edge of any type (it is a coverage measure: a CVE with five SEC\_\* edges and a CVE with one SEC\_\* edge both count once). “Mean SEC edges” is the *average count* of SEC\_\* edges per graph (a density measure: that same CVE with five SEC\_\* edges contributes five to the mean). “Mean sec. entities” sums the ten security entity categories (CVE\_ID, CWE\_ID, SOFTWARE, VERSION, VULN\_TYPE, ATTACK\_VECTOR, IMPACT, SEVERITY, REMEDIATION, CODE\_ELEMENT). The social-media counterpart of this table has been moved into Section 8, which now hosts the entire social-media baseline.

Two patterns stand out on the Megavul block:

- **Mistral generates the largest graphs** (988 mean nodes, against Gemma’s 640). The ratio of largest to smallest is around  $1.5\times$ .
- **Mistral has by far the most SEC\_\* edges per graph.** Mistral produces almost twice as many SEC\_\* edges per graph (98.8) as GPT (57.9), even though their mean graph sizes are similar. The commit-and-code text Mistral generates is denser in canonical-CVE phrasing than the other Megavul LLMs.

**Per-variant grid.** Table 8 extends the picture to the 15 Megavul datasets retained after the DeepSeek block was dropped (its *S\_cvss* variant fell back to the description text because the DeepSeek source ships with the CVSS-metrics summary empty for every row, which breaks per-variant parity across the four LLMs). The social-media counterpart of this grid is reported in Section 8 (Table 11).

One observation that recurs across the grid: the summary-only *S\_\** variants always produce smaller and lighter graphs than *ALL*, but they fire the security overlay almost as often. On GPT’s Megavul block, for example, *S\_url* produces graphs of 175 nodes / 597 edges (against *ALL*’s 848 / 2 858) but still has 97.2 % of CVEs producing a SEC\_\* edge against the *ALL* figure of 99.8 %. The LLM summaries on their own already preserve much of the canonical CVE language the security frontend keys off.

**Which security entities the LLMs surface.** Table 9 breaks the *ALL*-variant Megavul security entities down by category. The shared backbone is clear (every LLM surfaces some SOFTWARE and VERSION mentions, since those phrases are present in the canonical CVE description), but

| LLM     | Variant       | # graphs | Mean nodes | Mean edges | %CVEs $\geq 1$ SEC | Mean SEC edges |
|---------|---------------|----------|------------|------------|--------------------|----------------|
| GPT     | <i>D</i>      | 6,478    | 109        | 358        | 75.0 %             | 3.24           |
| GPT     | <i>S_url</i>  | 6,474    | 175        | 597        | 97.2 %             | 14.81          |
| GPT     | <i>S_code</i> | 6,302    | 397        | 1,263      | 95.5 %             | 15.53          |
| GPT     | <i>S_cvss</i> | 6,478    | 195        | 645        | 90.3 %             | 10.52          |
| GPT     | <i>ALL</i>    | 6,478    | 848        | 2,858      | 99.8 %             | 57.91          |
| Gemma   | <i>D</i>      | 6,478    | 109        | 358        | 75.0 %             | 3.24           |
| Gemma   | <i>S_url</i>  | 6,478    | 172        | 537        | 97.4 %             | 11.79          |
| Gemma   | <i>S_code</i> | 6,478    | 233        | 727        | 94.8 %             | 10.65          |
| Gemma   | <i>S_cvss</i> | 6,478    | 145        | 432        | 89.5 %             | 7.46           |
| Gemma   | <i>ALL</i>    | 6,478    | 640        | 2,065      | 100.0 %            | 38.90          |
| Mistral | <i>D</i>      | 6,478    | 109        | 358        | 75.0 %             | 3.24           |
| Mistral | <i>S_url</i>  | 6,478    | 208        | 645        | 97.7 %             | 17.67          |
| Mistral | <i>S_code</i> | 6,478    | 432        | 1,362      | 98.5 %             | 32.34          |
| Mistral | <i>S_cvss</i> | 6,362    | 263        | 811        | 87.7 %             | 9.41           |
| Mistral | <i>ALL</i>    | 6,478    | 988        | 3,201      | 99.9 %             | 98.83          |

Table 8: Per-LLM per-variant grid for the 15 retained Megavul datasets (DeepSeek dropped for missing CVSS-metrics content). *D* = description only, *S\_\** = summary-only, *ALL* = description plus combined summaries. Mean nodes and mean edges come from the cached graph tensors; the SEC\_\* statistics come from re-running the rule-only security pipeline over each labelled record. “%CVEs  $\geq 1$  SEC” is the *share* of CVEs whose graph contains at least one SEC\_\* edge of any type (coverage: 0–100 %, each CVE counted once). “Mean SEC edges” is the *average count* of SEC\_\* edges per graph (density: typically 3–99 here, a CVE with many SEC\_\* edges drives the mean up). The two columns therefore measure different things — a variant can have high coverage but low density (e.g. *S\_url* fires some SEC\_\* edge on 97.2 % of GPT graphs but averages only 14.8 per graph) or moderate coverage but very high density (e.g. Mistral *ALL* fires on 99.9 % of graphs and averages 98.8 per graph). On every Megavul row, the *D* variant has both the lowest coverage (75.0 %) and the lowest density (3.24 mean SEC edges) because the NVD description text in Megavul is short and code-focused, while every LLM-generated summary explicitly verbalises attack vectors, impacts, and software/version references that the security frontend keys off. The social-media counterpart appears as Table 11 in Section 8.

the per-LLM ordering of the action-relevant categories differs sharply. Mistral emits the most VULN\_TYPE, IMPACT, and CODE\_ELEMENT tags; GPT is the heaviest on ATTACK\_VECTOR and CODE\_ELEMENT.

| LLM     | CVE_ID | CWE_ID | SOFTWARE | VERSION | VULN_TYPE | ATTACK_VEC | IMPACT | SEVERITY | REMEDIATION | CODE_ELEM |
|---------|--------|--------|----------|---------|-----------|------------|--------|----------|-------------|-----------|
| GPT     | 0.05   | 0.00   | 1.98     | 1.69    | 1.35      | 7.61       | 3.02   | 1.76     | 1.03        | 3.87      |
| Gemma   | 0.05   | 0.00   | 1.71     | 1.58    | 2.58      | 4.02       | 3.14   | 2.70     | 0.86        | 2.76      |
| Mistral | 0.18   | 0.01   | 2.84     | 1.77    | 3.63      | 6.95       | 3.90   | 3.00     | 1.58        | 5.21      |

Table 9: Mean count of each security entity category per graph on the Megavul *ALL* variant. CWE\_ID is almost never extracted by the regex because the LLM summaries rarely quote the literal CWE identifier pattern; the CVE\_ID counts are also low because the commit-based text uses the commit hash rather than the formal CVE identifier. ATTACK\_VECTOR, IMPACT, SEVERITY, and CODE\_ELEMENT are the categories that vary most between LLMs and that the model’s exploitation-prediction task cares about. The social-media counterpart appears in Section 8 (Table 12).

**Which SEC\_\* edges fire.** Table 10 reports the mean count of each typed SEC\_\* edge per Megavul graph on the *ALL* variant. CAUSES dominates (reflecting the high IMPACT entity count) and LOCATED\_IN is the second most common edge because the commit summaries mention specific code constructs more often than free-text advisories do.

| LLM     | AFF  | HAS_V | CLASS | LOC_IN | EXPL | CAUSES | MITIG | USES | THREAT | HAS_S |
|---------|------|-------|-------|--------|------|--------|-------|------|--------|-------|
| GPT     | 0.12 | 4.05  | 0.00  | 6.62   | 0.37 | 31.27  | 0.05  | 0.19 | 15.15  | 0.08  |
| Gemma   | 0.10 | 3.22  | 0.00  | 8.68   | 0.18 | 19.35  | 0.04  | 0.12 | 7.09   | 0.12  |
| Mistral | 0.66 | 5.57  | 0.00  | 23.81  | 1.18 | 45.71  | 0.31  | 0.87 | 20.13  | 0.59  |

Table 10: Mean count of each typed SEC\_\* edge per graph on the Megavul *ALL* variant. Abbreviations: AFF = AFFECTS, HAS\_V = HAS\_VERSION, CLASS = CLASSIFIED\_AS, LOC\_IN = LOCATED\_IN, EXPL = EXPLOITED\_BY, MITIG = MITIGATED\_BY, USES = USES\_FUNCTION, THREAT = THREATENS, HAS\_S = HAS\_SEVERITY. CLASSIFIED\_AS is near zero because CWE\_IDs are rarely written in the descriptions or summaries; CAUSES, LOCATED\_IN, and THREATENS together account for over 75% of the security overlay on Megavul because the commit text references code constructs, impact outcomes, and attack vectors more explicitly than free-text advisories do. The social-media counterpart appears as Table 13 in Section 8.

**Why this matters for downstream performance.** The PR-AUC ranking inside the Megavul block of the earlier ablation already echoes these per-LLM differences. Gemma’s Megavul graphs carry the smallest security overlay (mean of 38.90 SEC\_\* edges per graph on *ALL*), so its model has the shortest auditable chain back to the text and the weakest SEC view to learn from. Mistral’s Megavul graphs are the largest and densest (988 nodes, 3,201 edges, 98.83 SEC\_\* edges per graph), so the security view contributes the most clearly there. A reasonable next step for the methodology section of any paper is to acknowledge this explicitly: the security frontend is not a free win that is uniform across LLM summarisers; it is a feature whose contribution scales with how much canonical CVE language the LLM keeps in its output. The social-media corpus (Section 8) reproduces this trend in the same direction but with different absolute magnitudes, dominated by the new *SMP* variant.

## 8 Social-media variant comparison (15 retrained runs)

A second pass over the social-media corpus retrained every LLM on a refreshed five-variant set, replacing the earlier *S\_all* (LLM summary of all sources) with *SMP*, the raw social-media post

text itself (truncated to a 16 KB budget; see Section 3). This addresses a reviewer concern: the LLM-generated summaries might be either compressing real signal away or hallucinating it in. *SMP* sends the underlying post straight through the pipeline without any LLM in the loop, so the comparison says *which text source actually helps the GNN predict exploitation*.

The retrained matrix covers three LLMs (GPT, Gemma, Mistral) crossed with five variants:

- *D* – the NVD description only.
- *SMP* – the raw social-media post only (no LLM summarisation).
- *S\_git* – the LLM’s summary of GitHub references found for the CVE.
- *S\_cvss* – the LLM’s summary of the CVSS metric narrative.
- *ALL* – the NVD description plus the three LLM summary columns combined.

DeepSeek is omitted from this matrix. Its source ships with the CVSS-metrics summary empty for every row, which breaks the *S\_cvss* variant and prevents a clean five-variant comparison; the other three LLMs are complete. Every run uses the same training schedule as Section 6 (multi-view GGNN, hybrid graph + tabular head, soft-EPSS regression target, EPSS feature removed from the tabular branch to prevent leakage).

## 8.1 Graph characterisation of the 15 retrained datasets

Before turning to the downstream metrics it is worth documenting what the GGNN actually consumes. The 15 labelled corpora were re-scanned with the same rule-only security pipeline used in Section 7, this time reading the per-CVE labelled records directly (the per-run graph caches are deleted after evaluation by the disk-cleanup hook described in Section 3).

**Cross-LLM sanity check.** On the description-only variant (*D*) every social-media graph is built from the same NVD description text, so the per-LLM numbers must match exactly. They do: every *D* run reports 110.348 mean nodes and 373.904 mean edges per graph, 4.604 mean SEC\_\* edges, and 4.884 mean security entities. The *SMP* variant is also byte-identical across LLMs because it bypasses the LLM and feeds the raw social-media post directly into the pipeline; all three *SMP* runs report 763.598 nodes, 3,374.523 edges, 246.815 SEC\_\* edges, and 21.212 security entities per graph. This confirms the methodology: any cross-LLM difference on *S\_git*, *S\_cvss*, and *ALL* is genuinely the LLM summariser’s output, not pipeline noise.

**Headline numbers per (LLM, variant).** Table 11 reports the per-variant graph size, the security-overlay density, and the processed-CVE count for every social-media run. *SMP* is uniformly the densest variant on every LLM (it is the same input on each row, after all) and 7× heavier on nodes than *D*. *ALL* comes in a clear second because it merges the three LLM summary columns with the description; the gap from *ALL* to *S\_cvss* or *S\_git* is large but not as dramatic as the *SMP* jump.

**Which security entities the LLMs surface.** Table 12 reports the mean count of each security entity category on the *ALL* variant. The shared backbone is the same as the Megavul side: every LLM surfaces SOFTWARE and VERSION mentions. Mistral is the heaviest on CVE\_ID, SOFTWARE, and SEVERITY; GPT is the heaviest on ATTACK\_VECTOR; Gemma is the lightest across the board, which mirrors its smaller and lighter graphs.

**Which SEC\_\* edges fire.** Table 13 reports the mean count of each typed SEC\_\* edge per graph on the *ALL* variant. The dominant edges on the social-media corpus are THREATENS,

| LLM     | Variant       | # graphs | Mean nodes | Mean edges | Mean SEC entities | Mean SEC edges |
|---------|---------------|----------|------------|------------|-------------------|----------------|
| GPT     | <i>D</i>      | 9,218    | 110.3      | 373.9      | 4.88              | 4.60           |
| GPT     | <i>SMP</i>    | 9,212    | 763.6      | 3,374.5    | 21.21             | 246.82         |
| GPT     | <i>S_git</i>  | 2,246    | 465.4      | 1,515.0    | 10.13             | 14.90          |
| GPT     | <i>S_cvss</i> | 9,218    | 188.5      | 606.6      | 5.20              | 0.77           |
| GPT     | <i>ALL</i>    | 9,218    | 595.8      | 2,094.5    | 19.32             | 57.70          |
| Gemma   | <i>D</i>      | 9,218    | 110.3      | 373.9      | 4.88              | 4.60           |
| Gemma   | <i>SMP</i>    | 9,212    | 763.6      | 3,374.5    | 21.21             | 246.82         |
| Gemma   | <i>S_git</i>  | 2,381    | 186.4      | 573.8      | 5.79              | 4.64           |
| Gemma   | <i>S_cvss</i> | 9,218    | 140.3      | 412.2      | 2.87              | 0.21           |
| Gemma   | <i>ALL</i>    | 9,218    | 441.1      | 1,430.6    | 14.19             | 31.38          |
| Mistral | <i>D</i>      | 9,218    | 110.3      | 373.9      | 4.88              | 4.60           |
| Mistral | <i>SMP</i>    | 9,212    | 763.6      | 3,374.5    | 21.21             | 246.82         |
| Mistral | <i>S_git</i>  | 2,388    | 265.2      | 838.4      | 8.45              | 11.76          |
| Mistral | <i>S_cvss</i> | 9,016    | 265.6      | 811.9      | 5.83              | 0.33           |
| Mistral | <i>ALL</i>    | 9,218    | 652.9      | 2,150.6    | 19.85             | 57.53          |

Table 11: Per-LLM per-variant graph characterisation for the 15 retrained social-media datasets. “# graphs” is the number of labelled CVEs that produced a graph (drops are caused by empty source columns: *SMP* loses 6 CVEs to empty social-media post fields; *S\_git* only applies to CVEs with a non-empty GitHub-references summary; Mistral *S\_cvss* loses 202 rows). The *D* and *SMP* rows are byte-identical across LLMs because they bypass the LLM. Counts come from re-running the rule-only security pipeline directly on each labelled record.

| LLM     | CVE_ID | CWE_ID | SOFTWARE | VERSION | VULN_TYPE | ATTACK_VEC | IMPACT | SEVERITY | REMED | CODE_ELEM |
|---------|--------|--------|----------|---------|-----------|------------|--------|----------|-------|-----------|
| GPT     | 0.62   | 0.02   | 2.82     | 2.20    | 0.78      | 5.90       | 2.00   | 1.61     | 1.46  | 1.91      |
| Gemma   | 0.73   | 0.02   | 2.74     | 1.84    | 1.03      | 2.73       | 1.58   | 1.77     | 0.88  | 0.88      |
| Mistral | 1.09   | 0.02   | 3.24     | 2.21    | 1.31      | 3.84       | 1.76   | 3.52     | 1.56  | 1.28      |

Table 12: Mean count of each security entity category per graph on the social-media *ALL* variant. The *CWE\_ID* counts are tiny because the LLM summaries rarely quote the literal CWE identifier string. *ATTACK\_VECTOR*, *IMPACT*, *SEVERITY*, and *CODE\_ELEMENT* are the categories that vary most between LLMs and that the exploitation-prediction task cares about.

CAUSES, and HAS\_VERSION, in line with the high ATTACK\_VECTOR and IMPACT entity counts. Mistral fires roughly twice as many AFFECTS and HAS\_SEVERITY edges as the other two LLMs because its summaries quote software and severity labels more verbatim.

| LLM     | AFF  | HAS_V | CLASS | LOC_IN | EXPL | CAUSES | MITIG | USES | THREAT | HAS_S |
|---------|------|-------|-------|--------|------|--------|-------|------|--------|-------|
| GPT     | 2.53 | 9.31  | 0.01  | 2.45   | 4.21 | 16.93  | 1.05  | 1.44 | 18.53  | 1.24  |
| Gemma   | 2.84 | 7.56  | 0.02  | 1.42   | 2.09 | 6.58   | 0.78  | 0.70 | 8.00   | 1.39  |
| Mistral | 5.33 | 11.53 | 0.02  | 2.70   | 4.53 | 11.37  | 2.00  | 1.61 | 13.63  | 4.81  |

Table 13: Mean count of each typed SEC\_\* edge per graph on the social-media *ALL* variant. Abbreviations as in Table 10. THREATENS dominates here because the attack-vector phrases that produce it (remote, network, attacker) recur in social-media chatter constantly. CLASSIFIED\_AS is near zero because CWE\_IDs are rarely written verbatim. The numbers for *SMP* (not shown) are uniformly much larger across all edge types because the raw 16 KB post text contains many more sentences than any LLM summary; *SMP* averages 246.8 SEC\_\* edges per graph against *ALL*’s 31–58.

**How the graphs map to the downstream metrics.** The graph characterisation explains two things that show up later in the downstream tables:

- *SMP* produces by far the largest and most security-dense graphs (246.8 SEC\_\* edges per graph, against *ALL*’s 31–58 and *D*’s 4.6). This is consistent with *SMP* delivering the highest ROC-AUC and recall on every LLM: more SEC\_\* anchor points give the GNN more lift on the long tail of positive CVEs. The flip side is that the extra mass is noisier (raw post text, no LLM curation), which is why precision is lower and Brier is higher on *SMP*.
- *S\_cvss* produces the lightest security overlay of all (0.2–0.8 mean SEC\_\* edges per graph) even though its node and edge counts are healthy. The CVSS-metric summary strips out almost every natural-language cue that the security frontend keys off, leaving the model with effectively only the syntactic backbone. This explains why its PR-AUC collapses to 0.49–0.55 in Table 16.

**Test-set composition (verified from the saved per-run test results).** The *D*, *S\_cvss*, and *ALL* variants evaluate on the full 1 385-CVE test split (positive prevalence 15.5 %). *SMP* evaluates on 1 383 CVEs because five CVEs ship with an empty social-media post field in the source data and are skipped. *S\_git* only evaluates on the subset of CVEs that have a non-empty GitHub-references summary (339–360 test CVEs, around 24–26 % of the full split), so its absolute numbers should be compared against each other rather than against the other variants. Table 14 lists every run’s test-set size.

**Headline ranking: test PR-AUC.** Table 15 is the per-LLM  $\times$  per-variant grid of test PR-AUC. Higher is better; bold marks the best variant within each row.

**Full six-metric grid.** Table 16 expands the comparison to the remaining metrics. Brier score is lower-is-better; the other five metrics are higher-is-better. The two pieces of evidence that change the story versus Table 15 alone:

- ROC-AUC actually crowns *SMP* for every single LLM. *D* and *ALL* win on PR-AUC by very small margins on two of the three LLMs, but *SMP* consistently delivers the highest ROC-AUC (0.9499–0.9572) and the highest recall.
- Precision and Brier favour the description-based variants (*D* and *ALL*) more clearly. *D* achieves 0.83–0.84 precision and the lowest Brier of 0.049–0.052, against *SMP*’s 0.67–0.73 precision and

| LLM     | Variant | Test CVEs | Positives | Prevalence |
|---------|---------|-----------|-----------|------------|
| GPT     | D       | 1 385     | 215       | 15.5 %     |
| GPT     | SMP     | 1 383     | 214       | 15.5 %     |
| GPT     | S_git   | 339       | 58        | 17.1 %     |
| GPT     | S_cvss  | 1 385     | 215       | 15.5 %     |
| GPT     | ALL     | 1 385     | 215       | 15.5 %     |
| Gemma   | D       | 1 385     | 215       | 15.5 %     |
| Gemma   | SMP     | 1 383     | 214       | 15.5 %     |
| Gemma   | S_git   | 360       | 59        | 16.4 %     |
| Gemma   | S_cvss  | 1 385     | 215       | 15.5 %     |
| Gemma   | ALL     | 1 385     | 215       | 15.5 %     |
| Mistral | D       | 1 385     | 215       | 15.5 %     |
| Mistral | SMP     | 1 383     | 214       | 15.5 %     |
| Mistral | S_git   | 359       | 59        | 16.4 %     |
| Mistral | S_cvss  | 1 354     | 213       | 15.7 %     |
| Mistral | ALL     | 1 385     | 215       | 15.5 %     |

Table 14: Test-set composition for the 15 retrained social-media runs. Mistral *S\_cvss* loses 31 CVEs because the Mistral source ships with an empty CVSS-metrics summary on a handful of rows.

| LLM     | D             | SMP           | S_git  | S_cvss | ALL           |
|---------|---------------|---------------|--------|--------|---------------|
| GPT     | <b>0.8394</b> | 0.8321        | 0.7553 | 0.4916 | 0.7878        |
| Gemma   | 0.8312        | 0.8429        | 0.6817 | 0.5513 | <b>0.8471</b> |
| Mistral | 0.8406        | <b>0.8408</b> | 0.5992 | 0.5303 | 0.8317        |

Table 15: Test PR-AUC by LLM and variant for the 15-run social-media ablation. Bold = best per row. The winning variant differs per LLM (D for GPT, ALL for Gemma, SMP for Mistral) but the spread is tight: the within-row gap from best to second-best is at most 0.0073 (Mistral) and 0.0042 (Gemma).

0.055–0.066 Brier. The model trained on raw social-media text is wider (catches more positives but with more false positives), the model trained on descriptions is narrower.

| LLM     | Variant | PR-AUC        | ROC-AUC       | F1            | Precision     | Recall        | Brier ↓       |
|---------|---------|---------------|---------------|---------------|---------------|---------------|---------------|
| GPT     | D       | <b>0.8394</b> | 0.9400        | <b>0.8095</b> | 0.8293        | 0.7907        | <b>0.0493</b> |
| GPT     | SMP     | 0.8321        | <b>0.9549</b> | 0.7590        | 0.6655        | <b>0.8832</b> | 0.0657        |
| GPT     | S_git   | 0.7553        | 0.9250        | 0.8065        | 0.7576        | 0.8621        | 0.0600        |
| GPT     | S_cvss  | 0.4916        | 0.8384        | 0.4541        | 0.5419        | 0.3907        | 0.1051        |
| GPT     | ALL     | 0.7878        | 0.9390        | 0.7621        | 0.6962        | 0.8419        | 0.0661        |
| Gemma   | D       | 0.8312        | 0.9397        | <b>0.8096</b> | 0.8400        | 0.7814        | <b>0.0508</b> |
| Gemma   | SMP     | 0.8429        | <b>0.9572</b> | 0.7699        | 0.7311        | <b>0.8131</b> | 0.0551        |
| Gemma   | S_git   | 0.6817        | 0.8933        | 0.5275        | 0.7500        | 0.4068        | 0.0974        |
| Gemma   | S_cvss  | 0.5513        | 0.8638        | 0.5673        | 0.5871        | 0.5488        | 0.1005        |
| Gemma   | ALL     | <b>0.8471</b> | 0.9476        | 0.6947        | <b>0.8732</b> | 0.5767        | 0.0573        |
| Mistral | D       | 0.8406        | 0.9402        | <b>0.7873</b> | 0.8299        | 0.7488        | <b>0.0519</b> |
| Mistral | SMP     | <b>0.8408</b> | <b>0.9499</b> | 0.7603        | 0.7068        | <b>0.8224</b> | 0.0599        |
| Mistral | S_git   | 0.5992        | 0.9036        | 0.1231        | 0.6667        | 0.0678        | 0.1311        |
| Mistral | S_cvss  | 0.5303        | 0.8497        | 0.5595        | 0.5270        | 0.5962        | 0.1054        |
| Mistral | ALL     | 0.8317        | 0.9466        | 0.7059        | <b>0.8302</b> | 0.6140        | 0.0566        |

Table 16: Full six-metric grid for the 15 retrained social-media runs. Bold = best within each LLM block. PR-AUC, ROC-AUC, F1, Precision and Recall are higher-is-better; Brier is lower-is-better. SMP wins on ROC-AUC and Recall across all three LLMs; D wins on F1 and Brier across all three; PR-AUC and Precision are split per LLM.

### What the numbers say about each text source.

- **SMP earns its keep.** The raw social-media post on its own matches or beats the NVD description on the ranking metrics (it wins on ROC-AUC for every LLM and is within 0.0073 PR-AUC of the best on two of three). This is the headline finding for the reviewers who asked whether the post text actually carried signal: it does, and it does not need an LLM summariser in front of it.
- **The S\_cvss variant is the clear loser across the board.** PR-AUC collapses to 0.49–0.55 on every LLM, more than 0.28 below the corresponding *D* score. The CVSS-metric summary strips the natural-language cues that the security frontend keys off (and that SecBERT embeds), leaving the GNN with little to work with.
- **S\_git lags but on a different test set.** *S\_git* evaluates on the 339–360 CVEs that actually have GitHub references, with a slightly higher positive prevalence (16–17%). Its lower PR-AUC is consistent with this smaller, harder slice rather than with the GitHub summary being weak in absolute terms.
- **ALL is the safest single-variant pick.** It is never the worst and is the best for Gemma; for GPT and Mistral it is within 0.05 PR-AUC of the per-row winner while keeping high precision and a small Brier. If the deployment target is one variant per LLM rather than per-LLM tuning, ALL is the pragmatic default.

**Verdict.** The retrained matrix vindicates the new *SMP* variant and disqualifies *S\_cvss*. The description-only baseline (*D*) remains a remarkably strong single-text-source option, especially for precision-sensitive scoring; *ALL* is the safest combined baseline. No single text source dominates on every metric; the trade-off is consistently *recall-and-ROC* (SMP) versus *precision-and-Brier* (D / ALL). For the paper, this is exactly the kind of clean ablation table the reviewers were asking for.